Trabalho Gerência de Configuração:
Desenvolvimento, Métricas e IA no Ecossistema de Software e de TI

Qualidade e Revisão de Código (Code Review)

Artur Dias Cousseau, Guilherme Augusto Giehl Garbin, Matheus Andreazza Pörsch e Miguel Valentini

UCS

FUNDAÇÃO UNIVERSIDADE DE CAXIAS DO SUL

NOSSA TESE

PERGUNTA DO TEMA: REVISAR CÓDIGO É RELEVANTE NO CONTEXTO ATUAL?

Sim, como uma camada de garantia da qualidade, junto a outras. Com a IA, o que muda é onde a atenção humana precisa estar.

01

É uma camada entre outras

Testes, linters, análise estática e CI/CD automatizam as verificações repetíveis.

02

O humano fica com o julgamento

A revisão humana avalia o contexto, a intenção da mudança e as decisões de projeto.

03

A atenção vira gargalo

À medida que a IA aumenta o volume de código, a atenção dos revisores pode se tornar o limite.

04

Direcionar a atenção

Mudanças menores e checagens automáticas levam a atenção ao que exige julgamento.

Por que as equipes revisam código umas das outras?

O estudo “Expectations, Outcomes, and Challenges of Modern Code Review“ foi realizado para responder essa pergunta.

Nos times da Microsoft, onde o estudo tomou lugar, o principal motivo declarado foi de achar falhas, tendo sido primeiro lugar por 44% dos gerentes de time e estando no top 3 por 78% dos programadores.

Porém ao analisar os comentários das revisões, notou-se que o encontro de falhas ficou em quarto lugar com 14% dos resultados, estando atrás de “melhorias de código”, “entendimento” e “comunicação”.

Revisão de código em Open Source Software

Dentro do estudo “Modern Code Reviews in Open-Source Projects: Which Problems Do They Fix?”, foi realizado uma análise de 1400 revisões de código em softwares de código aberto, para descobrir se o code review funciona em ambientes sem times fechados e coesos.

Eles também chegaram à mesma conclusão que o estudo anterior, porém eles foram mais longe e olharam para o código após a revisão, o quanto que os comentários realmente eram implementadas.

De 7 a 35% dos comentários não resultaram em mudanças no código e 10 a 22% das mudanças não vem de comentários explícitos.

AS DEFESAS JUNTO DO CODE REVIEW

DENTRO DO PR

01
CHECKS AUTOMÁTICOS

Build, testes e linter. Compila? Os testes passam?

>

02
REVISÃO POR IA

Um modelo lê o diff e comenta.

>

03
REVISÃO HUMANA

Contexto, intenção e decisões de projeto.

CI = verificar automaticamente cada mudança **CD** = entregar o que passou

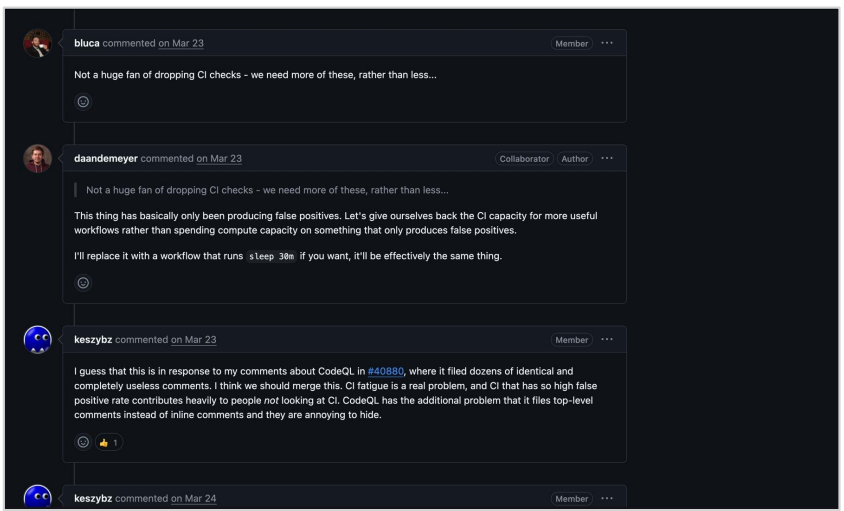
MAIS CAMADAS ≠ MAIS PROTEÇÃO



218 Alertas do CodeQL analisados

3 Correções úteis (em C++)

"dozens of identical and completely useless comments"
keszybz, mantenedor do systemd (PR #40880)



Debate sobre remover o CodeQL (PR #41277)

FONTES: SYSTEMD PR #41277 (ANÁLISE DO AUTOR) · PR #40880 (CITAÇÃO)

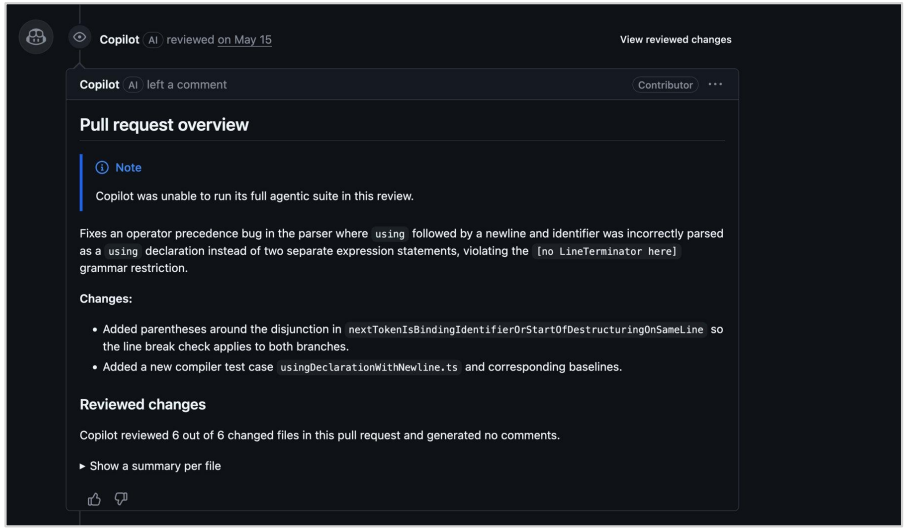
A IA NO PROCESSO: IA × HUMANO

NO PR

PR ABERTO > CHECKS AUTOMÁTICOS | REVISÃO POR IA | REVISÃO HUMANA > MERGE > DEPLOY

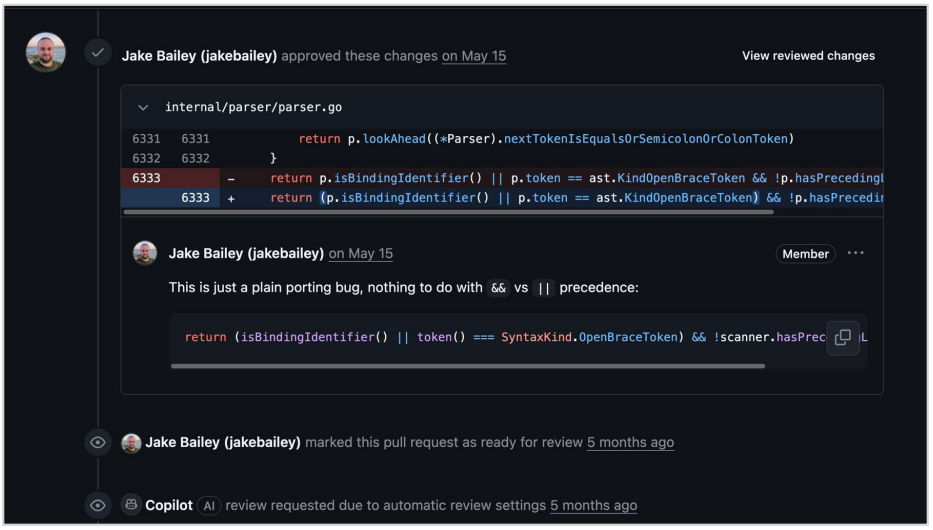
IA

Atribui o bug à precedência do Go. A IA revisora não comentou.



HUMANO

“plain porting bug”: compara com o TypeScript original.



FONTE: MICROSOFT/TYPESCRIPT-GO, PR #3869, 15/05/2026

O GARGALO VIROU A VERIFICAÇÃO



25% → 75%

Do código novo do Google é gerado por IA e revisado por engenheiros (2024 → 2026)

“Code generation is no longer the bottleneck — verification is.”

Gerar código não é mais o gargalo; verificar é. · Thoughtworks, 2026

PERGUNTA

Quanto dessa atenção o humano aguenta?

FONTES: S. PICHAI (OUT/2024 E ABR/2026) · THOUGHTWORKS, THE FUTURE OF SOFTWARE ENGINEERING, 2026

O Cansaço e a Produtividade de Código

Durante a criação de código, os programadores tendem a apresentar mais sintomas de cansaço conforme continuam a escrever sistematicamente, por causa de esforço repetitivo, conforme diz Tito Laneiro et. al.:

“A dissociação entre a vigilância e outros componentes da atenção sugerem um forte elo entre esta última variável, a tensão ou estresse e a fadiga. [...] É importante salientar que **geralmente a eficiência humana diminui em indivíduos que continuam a trabalhar, após terem consciência do seu estado de fadiga**“

Isso gera menor atenção ao código escrito, propiciando o surgimento de erros, sejam eles de sintaxe ou de algoritmo, reduzindo a qualidade de código e permitindo que erros persistam até o ambiente de produção.

Mais Descanso ↗ = **Mais produtividade** ↗

UCS

FUNDAÇÃO UNIVERSIDADE DE CAXIAS DO SUL

Burnout e Trabalho Remoto

Segundo os autores Beatriz Bandeira de Mello Aveiro e Carlos Manoel Lopes Rodrigues, em seu artigo “Burnout em desenvolvedores e a relação com o trabalho remoto”:

Um fenômeno importante é chamado burnout, descrito como **“aquilo que deixou de funcionar por exaustão de energia”**. O Ministério da Saúde define a síndrome de *burnout*, ou síndrome do esgotamento profissional, como um tipo de resposta prolongada a estressores emocionais e interpessoais crônicos no ambiente de trabalho.

Tempo Gasto em Recuperação e Testagem de Código

A área de TI naturalmente está sujeita a um maior esgotamento dos profissionais, pois trata-se de uma área com ritmo muito acelerado de entregas a fim de acompanhar as tendências de mercado e evoluções tecnológicas. Devido a carga de trabalho e a necessidade excessiva de concentração, os profissionais que são desenvolvedores acabam sendo atingidos em maior escala, isso porque a maioria dos programadores costumam passar horas seguidas na frente de seus computadores, incluindo madrugadas e finais de semana

Segundo o artigo “APLICAÇÕES DE MEDIDA DO TRABALHO EM ATIVIDADES DE PROGRAMAÇÃO DE COMPUTADORES”, de Paulo Roberto Feldmann, é natural que, em uma jornada de 8 h (equivalente a 480 minutos) cerca de 72 minutos sejam consumidos de forma a descansar, evitar a fadiga, desviar a atenção, necessidades fisiológicas, entre outras razões. Ou seja, cerca de 15% da jornada de trabalho de um desenvolvedor é utilizada para descanso inter tarefas, permitindo uma recuperação física e emocional. Além disso, segundo sua pesquisa, cerca de um terço do tempo de desenvolvimento de software é dirigido para a criação e uso de testes, objetivando a diminuição de erros e comportamentos não pretendidos antes do envio para o ambiente de produção.

	Dados obtidos p/ auto-avaliação		Dados obtidos p/ amostragem	
	hs	%s /Total	hs	%s /Total
Estudo	29	21	24	20
Codificação	34	25	31	26
Depuração	18	13	15	13
Testes	45	32	39	33
Documentação	12	9	9,5	8
Sub Total	138	100	118,5	100

Como resolver?

Com base na vivência dos integrantes do grupo, algumas propostas para reduzir a perda de atenção e de foco, resultando em maior qualidade de software e menos erros em produção, são:

- O uso da programação assistida com IA, funcionando como auxiliar e como dupla revisão de código;
- Geração de código primeiramente em blocos funcionais, com posterior união.
- Gerir melhor o tempo para cada tarefa, não alongando a produção de determinadas partes sem motivo claro.

Caso real: IA e revisão humana

- Saída do estagiário/testador
- Implementação da IA para revisar os códigos
- Como estamos revisando os códigos atualmente
- Responder a pergunta final: “revisar código é relevante no contexto atual?”

CASO REAL: IA E REVISÃO HUMANA

NA NOSSA EQUIPE

CÓDIGO PRONTO > ~~TESTADOR~~ | **CODEX (AUTOR)** | **REVISÃO HUMANA** > **MERGE** > **DEPLOY**

01
CONTEXTO
Testador saiu
Chefe: usar IA
Camada de testes trocada
Equipe pequena

>

02
FLUXO
Autor pede ao Codex
Abre o PR
Chefe revisa à mão
Autoverificação

>

03
O QUE MUDA
Mecânico: IA
Contexto: humano
Sem medição, só relato
Único revisor: atenção?

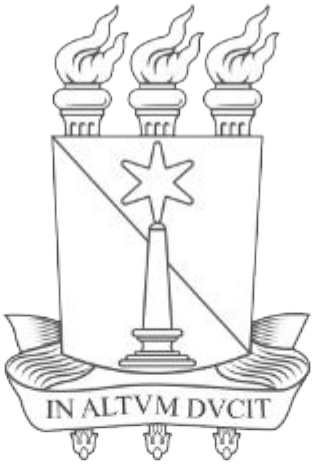
>

04
RESPOSTA
Revisar continua relevante
O que se revisa mudou
Proteger a atenção
Automatizar o mecânico

82% dos 873 desenvolvedores dizem que o feedback muda quando o revisor conhece os arquivos (Microsoft, 2013)

Revisar código é relevante no contexto atual?

FONTES: BACCHELLI & BIRD, ICSE 2013



FUNDAÇÃO UNIVERSIDADE
DE CAXIAS DO SUL

UCS HOSPITAL GERAL CETECUCS UCSfm